

Pandas

August 28, 2025

1 Manipulação e Combinação de Dados com Pandas

Pandas é uma ferramenta indispensável na ciência de dados, oferecendo estruturas de dados de alta performance e fáceis de usar, como a **Series** e o **DataFrame**. Com ela, podemos limpar, transformar, analisar e visualizar dados de forma eficiente.

Neste notebook, vamos explorar os seguintes tópicos em profundidade:

1. **Estruturas de Dados Fundamentais:** Como criar e entender **Series** e **DataFrames**.
2. **Manipulação de Colunas:** Técnicas para selecionar, adicionar e remover colunas de um **DataFrame**.
3. **Análise com groupby:** O poderoso mecanismo de “Dividir-Aplicar-Combinar” para realizar análises em subgrupos de dados.
4. **Combinação de DataFrames:** Os métodos essenciais **concat**, **merge** e **join** para unificar diferentes fontes de dados.

Vamos começar nossa jornada pelo universo da manipulação de dados com Pandas!

1.1 Parte 1: Estruturas de Dados Fundamentais em Pandas

Antes de manipularmos dados complexos, precisamos entender as duas estruturas principais do Pandas: a **Series** e o **DataFrame**.

1.1.1 1.1 Criando Séries (Series)

A **Series** é a estrutura de dados mais básica do Pandas. Pense nela como um array unidimensional, semelhante a uma coluna em uma planilha ou um vetor, mas com um diferencial importante: ela possui **rótulos de índice**. Isso a torna muito mais flexível que um array NumPy simples.

Vamos criar nossa primeira **Series**.

```
[1]: # Primeiro, vamos importar as bibliotecas que usaremos
import pandas as pd
import numpy as np

# Criando uma Series vazia
# Nota: O Pandas avisará que, no futuro, o tipo de dado padrão (dtype) para
# ↪Series vazias mudará de float64 para 'object'.
# É uma boa prática especificar o dtype se você souber qual será.
serie_vazia = pd.Series(dtype='object')
```

```

print("Uma Series Vazia:")
print(serie_vazia)

print("-" * 30)

# Criando uma Series a partir de um array NumPy
# Cada elemento da Series recebe um índice numérico padrão, começando do 0.
dados_array = np.array(['S', 'a', 'm', 'u', 'e', 'l'])
serie_letras = pd.Series(dados_array)
print("\nUma Series criada a partir de um array:")
print(serie_letras)

```

Uma Series Vazia:
Series([], dtype: object)

Uma Series criada a partir de um array:

```

0    S
1    a
2    m
3    u
4    e
5    l
dtype: object

```

1.1.2 1.2 Criando DataFrames

O DataFrame é a principal estrutura de dados do Pandas e o seu cavalo de batalha para a análise de dados. Ele é uma estrutura tabular bidimensional, semelhante a uma planilha do Excel ou uma tabela de um banco de dados SQL.

Principais características: - Possui colunas e linhas, ambas com seus próprios índices. - As colunas podem ter tipos de dados diferentes (números, texto, datas, etc.). - É extremamente flexível para manipulação de dados.

Existem várias maneiras de criar um DataFrame. Vamos ver as mais comuns.

```

[2]: # 1. Criando um DataFrame vazio
df_vazio = pd.DataFrame()
print("Um DataFrame Vazio:")
print(df_vazio)

print("-" * 30)

# 2. Criando um DataFrame a partir de uma lista
# O Pandas atribui um nome de coluna e índices numéricos automaticamente.
lista_simples = ['hoje', 'eh', 'terça', 'feira']
df_lista = pd.DataFrame(lista_simples)
print("\nDataFrame criado a partir de uma lista:")

```

```

print(df_lista)

print("-" * 30)

# 3. Criando um DataFrame a partir de um dicionário de listas (a forma mais
    ↳comum)
# As chaves do dicionário se tornam os nomes das colunas.
# Todas as listas (valores do dicionário) devem ter o mesmo comprimento.
dados_dict = {
    'Nome': ['Armando', 'Claudio', 'Alfredo', 'Gabriel', 'Fabricio'],
    'Idade': [36, 25, 19, 18, 47],
    'Cidade': ['Guarulhos', 'São Paulo', 'Santos', 'Guarulhos', 'São Paulo'],
    'Profissão': ['Advogado', 'Técnico', 'Advogado', 'Técnico', 'Advogado']
}

df = pd.DataFrame(dados_dict)
print("\nDataFrame criado a partir de um dicionário:")
print(df)

```

Um DataFrame Vazio:

Empty DataFrame

Columns: []

Index: []

DataFrame criado a partir de uma lista:

```

      0
0  hoje
1    eh
2  terça
3  feira

```

DataFrame criado a partir de um dicionário:

	Nome	Idade	Cidade	Profissão
0	Armando	36	Guarulhos	Advogado
1	Claudio	25	São Paulo	Técnico
2	Alfredo	19	Santos	Advogado
3	Gabriel	18	Guarulhos	Técnico
4	Fabricio	47	São Paulo	Advogado

1.2 Parte 2: Manipulação de Colunas em DataFrames

Agora que sabemos como criar DataFrames, vamos aprender a selecionar, adicionar e remover colunas, que são operações fundamentais no dia a dia da análise de dados.

1.2.1 2.1 Selecionando Colunas

Para trabalhar com os dados, precisamos primeiro acessá-los.

- Para selecionar **uma única coluna**, use a notação de colchetes `df['NomeDaColuna']`. O resultado será uma `Series`.
- Para selecionar **múltiplas colunas**, use colchetes duplos `df[['Coluna1', 'Coluna2']]`. O resultado será um novo `DataFrame`.

```
[3]: # Usando o DataFrame criado anteriormente
print("DataFrame Original:")
print(df)

print("-" * 35)

# Selecionando uma única coluna ('Nome')
serie_nomes = df['Nome']
print("\nSelecionando a coluna 'Nome' (resultado é uma Series):")
print(type(serie_nomes))
print(serie_nomes)

print("-" * 35)

# Selecionando múltiplas colunas ('Cidade' e 'Profissão')
sub_df = df[['Cidade', 'Profissão']]
print("\nSelecionando 'Cidade' e 'Profissão' (resultado é um DataFrame):")
print(type(sub_df))
print(sub_df)
```

DataFrame Original:

	Nome	Idade	Cidade	Profissão
0	Armando	36	Guarulhos	Advogado
1	Claudio	25	São Paulo	Técnico
2	Alfredo	19	Santos	Advogado
3	Gabriel	18	Guarulhos	Técnico
4	Fabricio	47	São Paulo	Advogado

Selecionando a coluna 'Nome' (resultado é uma Series):

```
<class 'pandas.core.series.Series'>
```

```
0    Armando
1    Claudio
2    Alfredo
3    Gabriel
4    Fabricio
```

```
Name: Nome, dtype: object
```

Selecionando 'Cidade' e 'Profissão' (resultado é um DataFrame):

```
<class 'pandas.core.frame.DataFrame'>
      Cidade Profissão
0  Guarulhos  Advogado
1   São Paulo   Técnico
2     Santos  Advogado
3  Guarulhos   Técnico
4   São Paulo  Advogado
```

1.2.2 2.2 Adicionando Novas Colunas

Enriquecer um DataFrame com novas informações é uma tarefa constante. Pandas oferece várias maneiras de fazer isso, cada uma com suas vantagens.

Método 1: Atribuição Direta Esta é a forma mais simples e direta. Basta atribuir uma lista, Series ou um valor escalar a um novo nome de coluna.

```
[4]: # Vamos criar uma cópia do nosso DataFrame para não alterar o original
df_copia = df.copy()

# Adicionando uma nova coluna 'Estado' com uma lista
# A lista precisa ter o mesmo número de elementos que o número de linhas do
↳ DataFrame
estados = ['SP', 'SP', 'SP', 'SP', 'SP']
df_copia['Estado'] = estados

print("DataFrame com a nova coluna 'Estado':")
print(df_copia)
```

DataFrame com a nova coluna 'Estado':

	Nome	Idade	Cidade	Profissão	Estado
0	Armando	36	Guarulhos	Advogado	SP
1	Claudio	25	São Paulo	Técnico	SP
2	Alfredo	19	Santos	Advogado	SP
3	Gabriel	18	Guarulhos	Técnico	SP
4	Fabricio	47	São Paulo	Advogado	SP

Método 2: DataFrame.insert() Diferente da atribuição direta, que sempre adiciona a coluna no final, o método insert() nos dá a liberdade de adicionar uma coluna em qualquer posição que desejarmos. Ele modifica o DataFrame original (operação *in-place*).

A sintaxe é: df.insert(loc, column_name, value) - loc: A posição (índice inteiro) onde a nova coluna será inserida. - column_name: O nome da nova coluna. - value: Os valores para a coluna.

```
[5]: # Adicionando uma coluna 'Salario' na posição 2 (após a coluna 'Idade')
salarios = [7500.00, 4200.00, 6800.00, 3900.00, 9100.00]
df_copia.insert(2, 'Salario', salarios)

print("DataFrame com a coluna 'Salario' inserida na posição 2:")
print(df_copia)
```

DataFrame com a coluna 'Salario' inserida na posição 2:

	Nome	Idade	Salario	Cidade	Profissão	Estado
0	Armando	36	7500.0	Guarulhos	Advogado	SP
1	Claudio	25	4200.0	São Paulo	Técnico	SP
2	Alfredo	19	6800.0	Santos	Advogado	SP
3	Gabriel	18	3900.0	Guarulhos	Técnico	SP
4	Fabricio	47	9100.0	São Paulo	Advogado	SP

```
[6]: # Adicionando uma coluna 'Anos de Experiencia' usando assign()
# O DataFrame original 'df_copia' não será alterado
anos_exp = [10, 5, 2, 1, 20]
df_novo = df_copia.assign(AnosDeExperiencia=anos_exp)

print("DataFrame original 'df_copia' (não foi modificado):")
print(df_copia)

print("\nNovo DataFrame retornado por assign():")
print(df_novo)

# Também podemos adicionar múltiplas colunas de uma vez
df_novo_multi = df_copia.assign(
    AnosDeExperiencia = [10, 5, 2, 1, 20],
    Avaliacao = [8.5, 9.0, 7.8, 8.1, 9.5]
)

print("\nAdicionando múltiplas colunas com assign():")
print(df_novo_multi)
```

DataFrame original 'df_copia' (não foi modificado):

	Nome	Idade	Salario	Cidade	Profissão	Estado
0	Armando	36	7500.0	Guarulhos	Advogado	SP
1	Claudio	25	4200.0	São Paulo	Técnico	SP
2	Alfredo	19	6800.0	Santos	Advogado	SP
3	Gabriel	18	3900.0	Guarulhos	Técnico	SP
4	Fabricio	47	9100.0	São Paulo	Advogado	SP

Novo DataFrame retornado por assign():

	Nome	Idade	Salario	Cidade	Profissão	Estado	AnosDeExperiencia
0	Armando	36	7500.0	Guarulhos	Advogado	SP	10
1	Claudio	25	4200.0	São Paulo	Técnico	SP	5
2	Alfredo	19	6800.0	Santos	Advogado	SP	2
3	Gabriel	18	3900.0	Guarulhos	Técnico	SP	1
4	Fabricio	47	9100.0	São Paulo	Advogado	SP	20

Adicionando múltiplas colunas com assign():

	Nome	Idade	Salario	Cidade	Profissão	Estado	AnosDeExperiencia	\
0	Armando	36	7500.0	Guarulhos	Advogado	SP	10	
1	Claudio	25	4200.0	São Paulo	Técnico	SP	5	

2	Alfredo	19	6800.0	Santos	Advogado	SP	2
3	Gabriel	18	3900.0	Guarulhos	Técnico	SP	1
4	Fabricio	47	9100.0	São Paulo	Advogado	SP	20

Avaliacao	
0	8.5
1	9.0
2	7.8
3	8.1
4	9.5

Método 4: DataFrame.loc O acessador `.loc` é usado principalmente para selecionar linhas e colunas por rótulo, mas também pode ser utilizado para criar novas colunas.

```
[7]: # Adicionando a coluna 'Bonus' usando .loc
# A sintaxe df.loc[:, 'NovaColuna'] seleciona todas as linhas (:) para a nova
#     ↳coluna.
bonus_valores = [1500, 800, 1300, 750, 1800]
df_copia.loc[:, 'Bonus'] = bonus_valores

print("DataFrame com a nova coluna 'Bonus' adicionada via .loc:")
print(df_copia)
```

DataFrame com a nova coluna 'Bonus' adicionada via `.loc`:

	Nome	Idade	Salario	Cidade	Profissão	Estado	Bonus
0	Armando	36	7500.0	Guarulhos	Advogado	SP	1500
1	Claudio	25	4200.0	São Paulo	Técnico	SP	800
2	Alfredo	19	6800.0	Santos	Advogado	SP	1300
3	Gabriel	18	3900.0	Guarulhos	Técnico	SP	750
4	Fabricio	47	9100.0	São Paulo	Advogado	SP	1800

1.2.3 2.3 Removendo Colunas

Assim como adicionamos, frequentemente precisamos remover colunas que não são mais necessárias para a análise. A principal ferramenta para isso é o método `.drop()`.

É crucial especificar o eixo (`axis=1` para colunas) ou usar o argumento `columns`. Outro parâmetro importante é o `inplace=True`, que modifica o `DataFrame` diretamente, sem a necessidade de reatribuí-lo a uma variável.

```
[8]: print("DataFrame antes de remover a coluna 'Bonus':")
print(df_copia)

# Removendo a coluna 'Bonus'. A operação é feita "in-place", modificando
#     ↳df_copia.
df_copia.drop(columns=['Bonus'], inplace=True)

print("\nDataFrame após remover a coluna 'Bonus':")
print(df_copia)
```

DataFrame antes de remover a coluna 'Bonus':

	Nome	Idade	Salario	Cidade	Profissão	Estado	Bonus
0	Armando	36	7500.0	Guarulhos	Advogado	SP	1500
1	Claudio	25	4200.0	São Paulo	Técnico	SP	800
2	Alfredo	19	6800.0	Santos	Advogado	SP	1300
3	Gabriel	18	3900.0	Guarulhos	Técnico	SP	750
4	Fabricio	47	9100.0	São Paulo	Advogado	SP	1800

DataFrame após remover a coluna 'Bonus':

	Nome	Idade	Salario	Cidade	Profissão	Estado
0	Armando	36	7500.0	Guarulhos	Advogado	SP
1	Claudio	25	4200.0	São Paulo	Técnico	SP
2	Alfredo	19	6800.0	Santos	Advogado	SP
3	Gabriel	18	3900.0	Guarulhos	Técnico	SP
4	Fabricio	47	9100.0	São Paulo	Advogado	SP

1.3 Parte 3: A Análise de Dados com groupby

A operação **groupby** é uma das mais poderosas do Pandas. Ela nos permite analisar dados de forma segmentada, aplicando funções a subgrupos específicos dentro do nosso conjunto de dados. É uma técnica amplamente utilizada em ciência de dados devido à sua eficiência.

1.3.1 3.1 O Conceito: Dividir-Aplicar-Combinar (Split-Apply-Combine)

A lógica do **groupby** pode ser resumida em três etapas:

1. **Dividir (Split):** O conjunto de dados é dividido em grupos com base em um ou mais critérios (os valores de uma coluna, por exemplo).
2. **Aplicar (Apply):** Uma função é aplicada a cada um desses grupos de forma independente. Pode ser uma agregação (**sum**, **mean**), uma transformação ou uma filtragem.
3. **Combinar (Combine):** Os resultados da aplicação da função em cada grupo são combinados em uma nova estrutura de dados (geralmente um **DataFrame** ou **Series**).

Ilustração Conceitual Vamos visualizar esse processo. Imagine que temos nosso **DataFrame** de funcionários e queremos calcular a média de idade por **Cidade**.

```
[9]: import pandas as pd
import matplotlib.pyplot as plt

# 1. Recriar o DataFrame original
dados_dict = {
    'Nome': ['Armando', 'Claudio', 'Alfredo', 'Gabriel', 'Fabricio'],
    'Idade': [36, 25, 19, 18, 47],
    'Cidade': ['Guarulhos', 'São Paulo', 'Santos', 'Guarulhos', 'São Paulo'],
    'Profissão': ['Advogado', 'Técnico', 'Advogado', 'Técnico', 'Advogado']
}
df = pd.DataFrame(dados_dict)

# 2. Criar a figura e os eixos para a plotagem
```

```

fig, ax = plt.subplots(figsize=(8, 4)) # Ajuste o tamanho conforme necessário

# 3. Ocultar os eixos (não queremos um gráfico, apenas a tabela)
ax.axis('tight')
ax.axis('off')

# 4. Adicionar um título à figura
fig.suptitle('IMAGEM CONCEITUAL 1: DataFrame Original', fontsize=16, y=0.85)

# 5. Criar a tabela a partir dos dados do DataFrame
tabela = ax.table(cellText=df.values, colLabels=df.columns, loc='center',
                  cellLoc='center')

# 6. Ajustar a aparência da tabela
tabela.auto_set_font_size(False)
tabela.set_fontsize(12)
tabela.scale(1.2, 1.2) # Aumenta o tamanho da tabela

# 7. Exibir a imagem
plt.show()

```

IMAGEM CONCEITUAL 1: DataFrame Original

Nome	Idade	Cidade	Profissão
Armando	36	Guarulhos	Advogado
Claudio	25	São Paulo	Técnico
Alfredo	19	Santos	Advogado
Gabriel	18	Guarulhos	Técnico
Fabricio	47	São Paulo	Advogado

[IMAGEM CONCEITUAL 1: DataFrame Original]

Tabela com as colunas ‘Nome’, ‘Idade’, ‘Cidade’, ‘Profissão’.

Etapas 1: DIVIDIR O Pandas cria “baldes” ou “caixas” para cada valor único na coluna ‘Cidade’.

[IMAGEM CONCEITUAL 2: Criação dos Grupos (Baldes)]

- Um balde para “Guarulhos”
- Um balde para “São Paulo”
- Um balde para “Santos”

As linhas correspondentes do `DataFrame` original são então colocadas em seus respectivos baldes.

[IMAGEM CONCEITUAL 3: Dados nos Grupos]

- Balde “Guarulhos”:
 - Linha 0 (Armando, 36)
 - Linha 3 (Gabriel, 18)
- Balde “São Paulo”:
 - Linha 1 (Claudio, 25)
 - Linha 4 (Fabricio, 47)
- Balde “Santos”:
 - Linha 2 (Alfredo, 19)

Etapa 2: APLICAR Aplicamos a função `mean()` (média) à coluna ‘Idade’ dentro de cada balde.

[IMAGEM CONCEITUAL 4: Aplicação da Função]

- Balde “Guarulhos”: `mean(36, 18) -> 27.0`
- Balde “São Paulo”: `mean(25, 47) -> 36.0`
- Balde “Santos”: `mean(19) -> 19.0`

Etapa 3: COMBINAR Os resultados são reunidos em uma nova `Series`, onde o índice são as chaves dos grupos (‘Cidade’).

[IMAGEM CONCEITUAL 5: Resultado Combinado]

[]:

[]:

```
[11]: import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as patches

# --- DADOS DE ENTRADA ---
# Recriar o DataFrame original para consistência
dados_dict = {
    'Nome': ['Armando', 'Claudio', 'Alfredo', 'Gabriel', 'Fabricio'],
    'Idade': [36, 25, 19, 18, 47],
    'Cidade': ['Guarulhos', 'São Paulo', 'Santos', 'Guarulhos', 'São Paulo'],
    'Profissão': ['Advogado', 'Técnico', 'Advogado', 'Técnico', 'Advogado']
}
df = pd.DataFrame(dados_dict)
cidades = df['Cidade'].unique()
cores = {'Guarulhos': 'lightblue', 'São Paulo': 'lightcoral', 'Santos': 'lightgreen'}

# --- IMAGEM 2: Criação dos Grupos (Baldes) ---
fig2, ax2 = plt.subplots(figsize=(8, 4))
ax2.set_xlim(0, 10)
ax2.set_ylim(0, 5)
```

```

ax2.axis('off')
fig2.suptitle('[IMAGEM CONCEITUAL 2: Criação dos Grupos (Balde)]',
    ↳ fontsize=16, y=0.9)

# Criando os "balde"
ax2.add_patch(patches.Rectangle((0.5, 1), 2.5, 2.5,
    ↳ facecolor=cores['Guarulhos'], edgecolor='black', linewidth=1.5))
ax2.text(1.75, 2.25, 'Guarulhos', ha='center', va='center', fontsize=14,
    ↳ weight='bold')

ax2.add_patch(patches.Rectangle((3.75, 1), 2.5, 2.5, facecolor=cores['São
    ↳ Paulo'], edgecolor='black', linewidth=1.5))
ax2.text(5, 2.25, 'São Paulo', ha='center', va='center', fontsize=14,
    ↳ weight='bold')

ax2.add_patch(patches.Rectangle((7, 1), 2.5, 2.5, facecolor=cores['Santos'],
    ↳ edgecolor='black', linewidth=1.5))
ax2.text(8.25, 2.25, 'Santos', ha='center', va='center', fontsize=14,
    ↳ weight='bold')

plt.show()

# --- IMAGEM 3: Dados nos Grupos ---
fig3, ax3 = plt.subplots(figsize=(8, 6))
ax3.set_xlim(0, 10)
ax3.set_ylim(0, 6)
ax3.axis('off')
fig3.suptitle('[IMAGEM CONCEITUAL 3: Dados nos Grupos]', fontsize=16, y=0.95)

# Balde de Guarulhos
dados_guarulhos = df[df['Cidade'] == 'Guarulhos'][['Nome', 'Idade']].
    ↳ to_string(index=False)
ax3.add_patch(patches.Rectangle((0.5, 1), 3, 4, facecolor=cores['Guarulhos'],
    ↳ alpha=0.3))
ax3.text(2, 5.5, 'Guarulhos', ha='center', weight='bold', fontsize=12)
ax3.text(2, 3, dados_guarulhos, ha='center', va='center', fontsize=11,
    ↳ fontfamily='monospace')

# Balde de São Paulo
dados_sp = df[df['Cidade'] == 'São Paulo'][['Nome', 'Idade']].
    ↳ to_string(index=False)
ax3.add_patch(patches.Rectangle((3.75, 1), 3, 4, facecolor=cores['São Paulo'],
    ↳ alpha=0.3))
ax3.text(5.25, 5.5, 'São Paulo', ha='center', weight='bold', fontsize=12)

```

```

ax3.text(5.25, 3, dados_sp, ha='center', va='center', fontsize=11,
        ↪fontfamily='monospace')

# Balde de Santos
dados_santos = df[df['Cidade'] == 'Santos'][['Nome', 'Idade']].
        ↪to_string(index=False)
ax3.add_patch(patches.Rectangle((7, 1), 2.5, 4, facecolor=cores['Santos'],
        ↪alpha=0.3))
ax3.text(8.25, 5.5, 'Santos', ha='center', weight='bold', fontsize=12)
ax3.text(8.25, 3, dados_santos, ha='center', va='center', fontsize=11,
        ↪fontfamily='monospace')

plt.show()

# --- IMAGEM 4: Aplicação da Função ---
fig4, ax4 = plt.subplots(figsize=(10, 5))
ax4.set_xlim(0, 12)
ax4.set_ylim(0, 6)
ax4.axis('off')
fig4.suptitle('[IMAGEM CONCEITUAL 4: Aplicação da Função]', fontsize=16, y=0.95)

ax4.text(6, 5, 'Função Aplicada: Média (mean) da Idade', ha='center',
        ↪fontsize=14, bbox=dict(facecolor='yellow', alpha=0.5))

# Seta indicando a aplicação
ax4.arrow(6, 4.5, 0, -1, head_width=0.3, head_length=0.3, fc='k', ec='k')

# Resultado por grupo
ax4.text(2, 2, 'Guarulhos\nmean(36, 18) = 27.0', ha='center', va='center',
        ↪fontsize=12,
        bbox=dict(boxstyle="round,pad=0.5", fc=cores['Guarulhos'], ec="b"))

ax4.text(6, 2, 'São Paulo\nmean(25, 47) = 36.0', ha='center', va='center',
        ↪fontsize=12,
        bbox=dict(boxstyle="round,pad=0.5", fc=cores['São Paulo'], ec="r"))

ax4.text(10, 2, 'Santos\nmean(19) = 19.0', ha='center', va='center',
        ↪fontsize=12,
        bbox=dict(boxstyle="round,pad=0.5", fc=cores['Santos'], ec="g"))

plt.show()

# --- IMAGEM 5: Resultado Combinado ---
# Calcular o resultado final com o Pandas

```

```

resultado_final = df.groupby('Cidade')['Idade'].mean().reset_index()
resultado_final.rename(columns={'Idade': 'Idade Média'}, inplace=True)
resultado_final['Idade Média'] = resultado_final['Idade Média'].round(1)

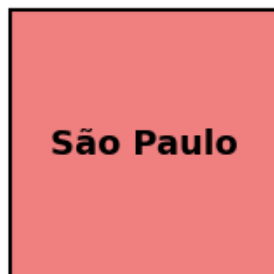
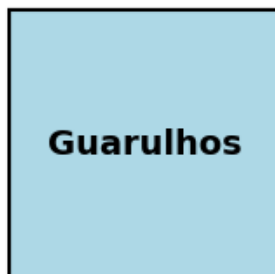
fig5, ax5 = plt.subplots(figsize=(6, 3))
ax5.axis('tight')
ax5.axis('off')
fig5.suptitle('[IMAGEM CONCEITUAL 5: Resultado Combinado]', fontsize=16, y=0.8)

# Criar a tabela com o resultado
tabela_final = ax5.table(cellText=resultado_final.values,
    ↪colLabels=resultado_final.columns, loc='center', cellLoc='center')
tabela_final.auto_set_font_size(False)
tabela_final.set_fontsize(12)
tabela_final.scale(1.2, 1.4)

plt.show()

```

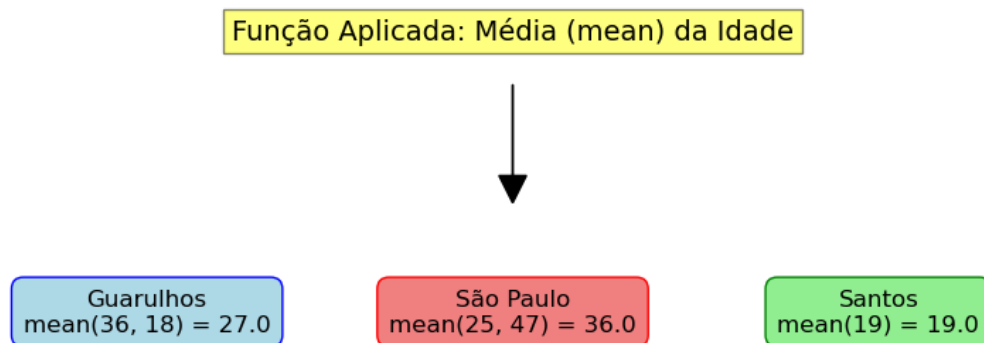
[IMAGEM CONCEITUAL 2: Criação dos Grupos (Balde)]



[IMAGEM CONCEITUAL 3: Dados nos Grupos]

Guarulhos	São Paulo	Santos																
<table><tr><th>Nome</th><th>Idade</th></tr><tr><td>Armando</td><td>36</td></tr><tr><td>Gabriel</td><td>18</td></tr></table>	Nome	Idade	Armando	36	Gabriel	18	<table><tr><th>Nome</th><th>Idade</th></tr><tr><td>Claudio</td><td>25</td></tr><tr><td>Fabricio</td><td>47</td></tr></table>	Nome	Idade	Claudio	25	Fabricio	47	<table><tr><th>Nome</th><th>Idade</th></tr><tr><td>Alfredo</td><td>19</td></tr></table>	Nome	Idade	Alfredo	19
Nome	Idade																	
Armando	36																	
Gabriel	18																	
Nome	Idade																	
Claudio	25																	
Fabricio	47																	
Nome	Idade																	
Alfredo	19																	

[IMAGEM CONCEITUAL 4: Aplicação da Função]



[IMAGEM CONCEITUAL 5: Resultado Combinado]

Cidade	Idade Média
Guarulhos	27.0
Santos	19.0
São Paulo	36.0

1.3.2 3.2 Criando Grupos (Dividir)

A primeira etapa é criar um objeto `DataFrameGroupBy` usando o método `.groupby()`. Este objeto contém todas as informações sobre os grupos.

```
[10]: # Agrupando nosso DataFrame 'df' pela coluna 'Cidade'
grupo_cidade = df.groupby('Cidade')

# O resultado é um objeto especial do Pandas
print("Tipo do objeto retornado por groupby():")
print(type(grupo_cidade))

# Podemos inspecionar os grupos formados com o atributo .groups
# Ele retorna um dicionário onde as chaves são os nomes dos grupos
# e os valores são os índices das linhas que pertencem a cada grupo.
print("\nGrupos formados e seus respectivos índices:")
print(grupo_cidade.groups)
```

Tipo do objeto retornado por `groupby()`:

```
<class 'pandas.core.groupby.generic.DataFrameGroupBy'>
```

Grupos formados e seus respectivos índices:

```
{'Guarulhos': [0, 3], 'Santos': [2], 'São Paulo': [1, 4]}
```

Também é possível agrupar por **múltiplas colunas**. Nesse caso, os grupos serão formados pelas combinações únicas de valores dessas colunas.

```
[ ]: # Agrupando por 'Cidade' e 'Profissão'
grupo_cidade_profissao = df.groupby(['Cidade', 'Profissão'])
```

```
# As chaves do dicionário de grupos agora são tuplas
print("Grupos formados por 'Cidade' e 'Profissão':")
print(grupo_cidade_profissao.groups)
```

1.3.3 3.3 Operações em Grupos (Aplicar-Combinar)

Uma vez que os grupos são criados, podemos aplicar uma variedade de operações.

Agregação Agregação é o processo de calcular uma estatística de resumo para cada grupo, como a soma, média, contagem, mínimo, máximo, etc. O resultado é uma estrutura de dados com os valores agregados, onde o índice corresponde aos nomes dos grupos.

A forma mais simples é chamar a função de agregação diretamente no objeto `groupby`.

```
[13]: # Calculando a idade média por cidade
media_idade_cidade = df.groupby('Cidade')['Idade'].mean()
print("Média de idade por cidade:")
print(media_idade_cidade)

print("\n" + "-"*40 + "\n")

# Calculando a soma das idades por profissão
soma_idade_profissao = df.groupby('Profissão')['Idade'].sum()
print("Soma das idades por profissão:")
print(soma_idade_profissao)
```

```
Média de idade por cidade:
Cidade
Guarulhos    27.0
Santos       19.0
São Paulo    36.0
Name: Idade, dtype: float64
```

```
Soma das idades por profissão:
Profissão
Advogado     102
Técnico       43
Name: Idade, dtype: int64
```

Para maior flexibilidade, usamos o método `.agg()` (ou `.aggregate()`). Ele nos permite: 1. Aplicar múltiplas funções de agregação de uma vez. 2. Aplicar funções diferentes a colunas diferentes.

```
[14]: # 1. Aplicando múltiplas funções a uma coluna
# Vamos agrupar por profissão e calcular a soma, média e desvio padrão da idade.
agg_multiplas_funcoes = df.groupby('Profissão')['Idade'].agg([np.sum, np.mean,
↳ np.std])
print("Múltiplas agregações na coluna 'Idade' por 'Profissão':")
```

```

print(agg_multiplas_funcoes)

print("\n" + "-"*50 + "\n")

# 2. Aplicando funções diferentes a colunas diferentes
# Vamos agrupar por cidade e calcular a média de Idade e o primeiro Nome em
↳ ordem alfabética.
agg_funcoes_diferentes = df.groupby('Cidade').agg(
    IdadeMedia=('Idade', 'mean'),
    Contagem=('Nome', 'count'),
    ProfissaoMaisComum=('Profissão', pd.Series.mode) # pd.Series.mode para a
↳ moda
)
print("Agregações diferentes para colunas diferentes por 'Cidade':")
print(agg_funcoes_diferentes)

```

Múltiplas agregações na coluna 'Idade' por 'Profissão':

	sum	mean	std
Profissão			
Advogado	102	34.0	14.106736
Técnico	43	21.5	4.949747

Agregações diferentes para colunas diferentes por 'Cidade':

	IdadeMedia	Contagem	ProfissaoMaisComum
Cidade			
Guarulhos	27.0	2	[Advogado, Técnico]
Santos	19.0	1	Advogado
São Paulo	36.0	2	[Advogado, Técnico]

Transformação Às vezes, não queremos resumir os dados, mas sim realizar um cálculo dentro de cada grupo e retornar um resultado com a **mesma forma do DataFrame original**. Para isso, usamos o método `.transform()`.

Um caso de uso comum é a padronização de dados (como o cálculo do Z-score) dentro de cada categoria. Vamos, por exemplo, centralizar a idade de cada pessoa em relação à média de sua profissão.

```

[15]: # A função lambda calcula, para cada idade (x), a diferença entre ela e a média
↳ de seu grupo.
# x representa a Series de idades de cada grupo.
centralizar_na_media = lambda x: x - x.mean()

df['Idade_Centralizada'] = df.groupby('Profissão')['Idade'].
↳ transform(centralizar_na_media)

print("DataFrame com a idade centralizada pela média da profissão:")

```

```
print(df)
```

```
# Note que o resultado tem o mesmo número de linhas que o df original
```

DataFrame com a idade centralizada pela média da profissão:

	Nome	Idade	Cidade	Profissão	Idade_Centralizada
0	Armando	36	Guarulhos	Advogado	2.0
1	Claudio	25	São Paulo	Técnico	3.5
2	Alfredo	19	Santos	Advogado	-15.0
3	Gabriel	18	Guarulhos	Técnico	-3.5
4	Fabricio	47	São Paulo	Advogado	13.0

Filtragem A filtragem permite remover grupos inteiros com base em uma condição. O método `.filter()` aplica uma função que retorna `True` ou `False` para cada grupo. Apenas os grupos para os quais a função retorna `True` são mantidos.

Por exemplo, vamos manter apenas as profissões que aparecem 2 ou mais vezes no nosso DataFrame.

```
[16]: # A função lambda verifica, para cada grupo (x), se o seu tamanho (len(x)) é
      ↪ maior ou igual a 2.
df_filtrado = df.groupby('Profissão').filter(lambda x: len(x) >= 2)

print("DataFrame filtrado, mantendo apenas profissões com 2 ou mais pessoas:")
print(df_filtrado)
```

DataFrame filtrado, mantendo apenas profissões com 2 ou mais pessoas:

	Nome	Idade	Cidade	Profissão	Idade_Centralizada
0	Armando	36	Guarulhos	Advogado	2.0
1	Claudio	25	São Paulo	Técnico	3.5
2	Alfredo	19	Santos	Advogado	-15.0
3	Gabriel	18	Guarulhos	Técnico	-3.5
4	Fabricio	47	São Paulo	Advogado	13.0

1.3.4 3.4 Iterando sobre Grupos

É possível iterar sobre um objeto `groupby`, o que gera pares de `(nome_do_grupo, dados_do_grupo)` a cada iteração.

```
[17]: # Iterando sobre os grupos de 'Cidade'
for cidade, grupo in df.groupby('Cidade'):
    print(f"--- Grupo da Cidade: {cidade} ---")
    print(grupo)
    print("\n")
```

--- Grupo da Cidade: Guarulhos ---

	Nome	Idade	Cidade	Profissão	Idade_Centralizada
0	Armando	36	Guarulhos	Advogado	2.0
3	Gabriel	18	Guarulhos	Técnico	-3.5

```

--- Grupo da Cidade: Santos ---
      Nome  Idade  Cidade Profissão  Idade_Centralizada
2  Alfredo    19   Santos  Advogado             -15.0

--- Grupo da Cidade: São Paulo ---
      Nome  Idade  Cidade Profissão  Idade_Centralizada
1  Claudio    25  São Paulo   Técnico             3.5
4  Fabricio   47  São Paulo  Advogado            13.0

```

1.3.5 3.5 Selecionando um Grupo Específico

Se você precisar acessar os dados de um único grupo específico, o método `.get_group()` é a ferramenta ideal.

```

[18]: # Agrupando por profissão
grupo_profissao = df.groupby('Profissão')

# Pegando apenas o grupo dos 'Advogados'
df_advogados = grupo_profissao.get_group('Advogado')

print("DataFrame contendo apenas o grupo 'Advogado':")
print(df_advogados)

```

```

DataFrame contendo apenas o grupo 'Advogado':
      Nome  Idade  Cidade Profissão  Idade_Centralizada
0  Armando    36  Guarulhos  Advogado             2.0
2  Alfredo    19   Santos  Advogado            -15.0
4  Fabricio   47  São Paulo  Advogado            13.0

```

1.4 Parte 4: Combinando DataFrames

Na análise de dados do mundo real, as informações raramente vêm em um único arquivo. Geralmente, precisamos combinar múltiplos `DataFrames` para criar um conjunto de dados unificado e mais rico. O Pandas oferece três funções principais para essa tarefa, cada uma adequada para um cenário diferente.

1. `pd.concat()`: Ideal para “empilhar” ou “colar” `DataFrames`, seja ao longo das linhas (um embaixo do outro) ou das colunas (um ao lado do outro).
2. `pd.merge()`: A função mais flexível e poderosa, implementa operações de junção no estilo de bancos de dados relacionais (como JOIN em SQL), combinando `DataFrames` com base em valores de colunas ou índices em comum.
3. `DataFrame.join()`: Uma forma mais conveniente para um caso de uso específico de `merge`, que é a junção de `DataFrames` com base em seus índices.

Vamos explorar cada uma delas em detalhes.

1.4.1 4.1 pd.concat(): Empilhando e Juntando Lado a Lado

A função `concat()` é a mais direta para combinar `DataFrames`. Pense nela como um processo de “costura” ou “colagem” de tabelas.

Concatenação Vertical (axis=0) Este é o comportamento padrão. Os `DataFrames` são empilhados um sobre o outro.

```
[19]: import matplotlib.pyplot as plt

# --- Criação da Figura e Eixos ---
# Usaremos dois eixos para mostrar o "antes" e o "depois"
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 6))
fig.suptitle('[IMAGEM CONCEITUAL: Concatenação Vertical]', fontsize=16,
             weight='bold', y=0.8)

# --- Eixo 1: DataFrames Iniciais ---
ax1.set_title('"Antes"')
ax1.axis('off')
ax1.set_xlim(0, 4)
ax1.set_ylim(0, 10)

# Dados para as tabelas
df_a_data = [['A', 1], ['B', 2]]
df_b_data = [['C', 3], ['D', 4]]
col_labels = ['Coluna 1', 'Coluna 2']

# Tabela 1 (DataFrame A)
tabela_a = ax1.table(cellText=df_a_data, loc='upper center',
                    colLabels=col_labels, cellLoc='center')
tabela_a.scale(1, 1.5)
tabela_a.set_fontsize(12)
ax1.text(2, 8, 'DataFrame A', ha='center', fontsize=12, weight='bold')

# Tabela 2 (DataFrame B)
tabela_b = ax1.table(cellText=df_b_data, loc='center', colLabels=col_labels,
                    cellLoc='center')
tabela_b.scale(1, 1.5)
tabela_b.set_fontsize(12)
ax1.text(2, 4.5, 'DataFrame B', ha='center', fontsize=12, weight='bold')

# Adicionando uma seta grande no meio da figura para indicar a transformação
fig.text(0.5, 0.5, r'$\rightarrow$', fontsize=40, ha='center', va='center')

# --- Eixo 2: DataFrame Resultante ---
ax2.set_title('"Depois"')
```

```

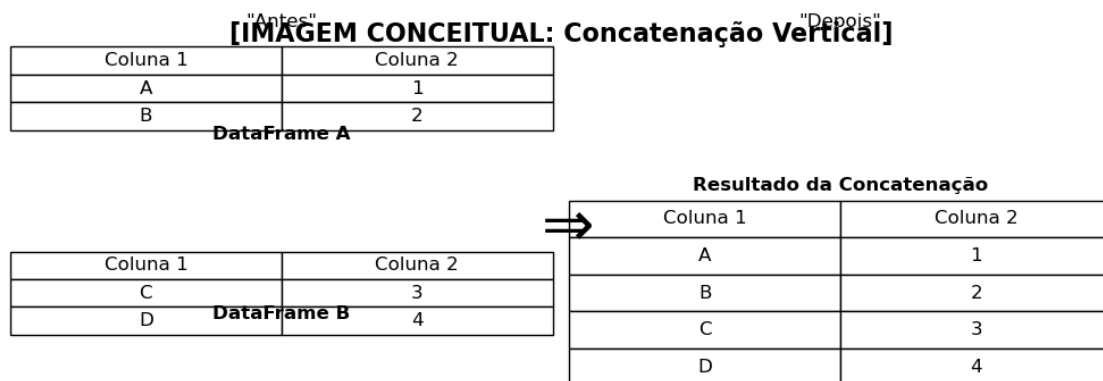
ax2.axis('off')
ax2.set_xlim(0, 4)
ax2.set_ylim(0, 10)

# Dados para a tabela final
df_c_data = [['A', 1], ['B', 2], ['C', 3], ['D', 4]]

# Tabela Final (DataFrame C)
tabela_c = ax2.table(cellText=df_c_data, loc='center', colLabels=col_labels,
                    ↪cellLoc='center')
tabela_c.scale(1, 2)
tabela_c.set_fontsize(12)
ax2.text(2, 7, 'Resultado da Concatenação', ha='center', fontsize=12,
        ↪weight='bold')

# Ajustar layout para evitar sobreposição de títulos
plt.tight_layout(rect=[0, 0, 1, 0.9])
plt.show()

```



[IMAGEM CONCEITUAL: Concatenação Vertical]

Dois DataFrames menores, um acima do outro, com setas apontando para um único DataFrame maior que contém as linhas de ambos.

[20]:

```

# Vamos criar dois DataFrames de exemplo
df1 = pd.DataFrame({
    'Nome': ['Armando', 'Claudio'],
    'Idade': [36, 25],

```

```

        'Cidade': ['Guarulhos', 'São Paulo']
    }, index=[0, 1])

df2 = pd.DataFrame({
    'Nome': ['Walter', 'Pablo'],
    'Idade': [36, 25],
    'Cidade': ['Guarulhos', 'São Paulo']
}, index=[5, 6]) # Índices diferentes para ilustrar

print("--- DataFrame 1 ---")
print(df1)
print("\n--- DataFrame 2 ---")
print(df2)

# Concatenando verticalmente
resultado_concat = pd.concat([df1, df2])

print("\n--- Resultado da Concatenação ---")
print(resultado_concat)
# Observe que os índices originais são mantidos por padrão.

```

```

--- DataFrame 1 ---
      Nome  Idade  Cidade
0  Armando    36  Guarulhos
1  Claudio    25  São Paulo

--- DataFrame 2 ---
      Nome  Idade  Cidade
5  Walter    36  Guarulhos
6   Pablo    25  São Paulo

--- Resultado da Concatenação ---
      Nome  Idade  Cidade
0  Armando    36  Guarulhos
1  Claudio    25  São Paulo
5   Walter    36  Guarulhos
6    Pablo    25  São Paulo

```

Frequentemente, após concatenar, queremos que o novo `DataFrame` tenha um índice contínuo e resetado. Para isso, usamos o parâmetro `ignore_index=True`.

Outra funcionalidade útil é o parâmetro `keys`, que cria um índice hierárquico, permitindo-nos rastrear a origem dos dados.

```

[21]: # Concatenando com ignore_index=True
resultado_reindexado = pd.concat([df1, df2], ignore_index=True)
print("--- Concatenação com ignore_index=True ---")
print(resultado_reindexado)

```

```
# Concatenando com keys para criar um índice hierárquico
resultado_hierarquico = pd.concat([df1, df2], keys=['turma_A', 'turma_B'])
print("\n--- Concatenação com keys ---")
print(resultado_hierarquico)

# Podemos usar o índice hierárquico para selecionar dados
print("\nSelecionando dados da 'turma_A':")
print(resultado_hierarquico.loc['turma_A'])
```

--- Concatenação com ignore_index=True ---

	Nome	Idade	Cidade
0	Armando	36	Guarulhos
1	Claudio	25	São Paulo
2	Walter	36	Guarulhos
3	Pablo	25	São Paulo

--- Concatenação com keys ---

	Nome	Idade	Cidade
turma_A 0	Armando	36	Guarulhos
1	Claudio	25	São Paulo
turma_B 5	Walter	36	Guarulhos
6	Pablo	25	São Paulo

Selecionando dados da 'turma_A':

	Nome	Idade	Cidade
0	Armando	36	Guarulhos
1	Claudio	25	São Paulo

Concatenação Horizontal (axis=1) Ao definir axis=1, o concat une os DataFrames lado a lado, alinhando-os pelos seus índices.

[IMAGEM CONCEITUAL: Concatenação Horizontal]

Dois DataFrames um ao lado do outro, com setas apontando para um DataFrame mais largo que contém as colunas de ambos.

```
[27]: import matplotlib.pyplot as plt
import pandas as pd

# Com certeza. Peço desculpas novamente pela dificuldade com o layout.
# Segue uma nova versão da imagem, agora com os DataFrames A e B na parte de
↳ cima
# e o resultado da concatenação horizontal embaixo, conforme sua solicitação.
# Este layout vertical deve eliminar qualquer sobreposição.

# --- Criação da Figura e Eixos ---
# Usaremos um único eixo e uma figura mais alta para o layout vertical
```

```

fig, ax = plt.subplots(figsize=(8, 8))
fig.suptitle('[IMAGEM CONCEITUAL: Concatenação Horizontal (Layout Vertical)]',
             ↪fontsize=16, weight='bold', y=0.95)
ax.axis('off')

# --- Seção Superior: DataFrames Iniciais ---
ax.text(0.5, 0.9, '"Antes"', ha='center', fontsize=14)

# Dados para as tabelas
df_a_data = [['36', 'Guarulhos'], ['25', 'São Paulo']]
df_b_data = [['Advogado', '7500'], ['Técnico', '4200']]
index_labels = ['Armando', 'Claudio']
col_labels_a = ['Idade', 'Cidade']
col_labels_b = ['Profissão', 'Salário']

# Tabela 1 (DataFrame A) - Posicionada na parte superior esquerda
tabela_a = ax.table(cellText=df_a_data,
                    rowLabels=index_labels,
                    colLabels=col_labels_a,
                    cellLoc='center',
                    bbox=[0.05, 0.65, 0.4, 0.2]) # bbox=[x, y, largura, altura]
tabela_a.auto_set_font_size(False)
tabela_a.set_fontsize(12)
ax.text(0.25, 0.86, 'DataFrame A', ha='center', fontsize=12, weight='bold')

# Tabela 2 (DataFrame B) - Posicionada na parte superior direita
tabela_b = ax.table(cellText=df_b_data,
                    rowLabels=index_labels,
                    colLabels=col_labels_b,
                    cellLoc='center',
                    bbox=[0.55, 0.65, 0.4, 0.2])
tabela_b.auto_set_font_size(False)
tabela_b.set_fontsize(12)
ax.text(0.75, 0.86, 'DataFrame B', ha='center', fontsize=12, weight='bold')

# --- Seção Intermediária: Seta ---
ax.text(0.5, 0.5, r'$\downarrow$', fontsize=40, ha='center', va='center',
       ↪color='blue')

# --- Seção Inferior: DataFrame Resultante ---
ax.text(0.5, 0.4, '"Depois"', ha='center', fontsize=14)

# Dados para a tabela final

```

```

df_c_data = [['36', 'Guarulhos', 'Advogado', '7500'], ['25', 'São Paulo', 'Técnico', '4200']]
col_labels_c = ['Idade', 'Cidade', 'Profissão', 'Salário']

# Tabela Final (DataFrame C)
tabela_c = ax.table(cellText=df_c_data,
                    rowLabels=index_labels,
                    colLabels=col_labels_c,
                    cellLoc='center',
                    bbox=[0.1, 0.15, 0.8, 0.2])
tabela_c.auto_set_font_size(False)
tabela_c.set_fontsize(12)
ax.text(0.5, 0.36, 'Resultado da Concatenação', ha='center', fontsize=12, weight='bold')

# Exibir a imagem
plt.show()

```

[IMAGEM CONCEITUAL: Concatenação Horizontal (Layout Vertical)]

"Antes"

	DataFrame A			DataFrame B	
	Idade	Cidade		Profissão	Salário
Armando	36	Guarulhos	Armando	Advogado	7500
Claudio	25	São Paulo	Claudio	Técnico	4200



"Depois"

Resultado da Concatenação

	Idade	Cidade	Profissão	Salário
Armando	36	Guarulhos	Advogado	7500
Claudio	25	São Paulo	Técnico	4200

```
[28]: # Vamos criar DataFrames que compartilham o mesmo índice
df_pessoal = pd.DataFrame({
    'Idade': [36, 25],
    'Cidade': ['Guarulhos', 'São Paulo']
}, index=['Armando', 'Claudio'])

df_profissional = pd.DataFrame({
    'Profissão': ['Advogado', 'Técnico'],
    'Salario': [7500, 4200]
}, index=['Armando', 'Claudio'])

print("--- Dados Pessoais ---")
print(df_pessoal)
print("\n--- Dados Profissionais ---")
print(df_profissional)

# Concatenando horizontalmente
resultado_horizontal = pd.concat([df_pessoal, df_profissional], axis=1)
print("\n--- Resultado da Concatenação Horizontal (axis=1) ---")
print(resultado_horizontal)
```

```
--- Dados Pessoais ---
```

	Idade	Cidade
Armando	36	Guarulhos
Claudio	25	São Paulo

```
--- Dados Profissionais ---
```

	Profissão	Salario
Armando	Advogado	7500
Claudio	Técnico	4200

```
--- Resultado da Concatenação Horizontal (axis=1) ---
```

	Idade	Cidade	Profissão	Salario
Armando	36	Guarulhos	Advogado	7500
Claudio	25	São Paulo	Técnico	4200

Controle de Alinhamento com join O que acontece quando os índices não se alinham perfeitamente em uma concatenação horizontal? O parâmetro `join` controla isso: - `join='outer'` (padrão): Mantém todos os índices de todos os `DataFrames` e preenche os valores ausentes com `NaN` (Not a Number). - `join='inner'`: Mantém apenas os índices que são comuns a **todos** os `DataFrames`.

```
[29]: # Criando um DataFrame com um índice diferente
df_bonus = pd.DataFrame({
    'Bonus': [1500, 800, 1300]
}, index=['Armando', 'Claudio', 'Alfredo']) # 'Alfredo' não existe nos outros
```

```

print("--- DataFrame de Bônus ---")
print(df_bonus)

# Concatenação com join='outer' (padrão)
outer_join_concat = pd.concat([df_pessoal, df_profissional, df_bonus], axis=1)
print("\n--- Concatenação com join='outer' ---")
print(outer_join_concat)
# Observe a linha para 'Alfredo' e os NaN's

# Concatenação com join='inner'
inner_join_concat = pd.concat([df_pessoal, df_profissional, df_bonus], axis=1,
    ↪join='inner')
print("\n--- Concatenação com join='inner' ---")
print(inner_join_concat)
# A linha 'Alfredo' foi descartada pois seu índice não estava presente em todos
    ↪os DataFrames.

```

--- DataFrame de Bônus ---

	Bonus
Armando	1500
Claudio	800
Alfredo	1300

--- Concatenação com join='outer' ---

	Idade	Cidade	Profissão	Salario	Bonus
Armando	36.0	Guarulhos	Advogado	7500.0	1500
Claudio	25.0	São Paulo	Técnico	4200.0	800
Alfredo	NaN	NaN	NaN	NaN	1300

--- Concatenação com join='inner' ---

	Idade	Cidade	Profissão	Salario	Bonus
Armando	36	Guarulhos	Advogado	7500	1500
Claudio	25	São Paulo	Técnico	4200	800

1.4.2 4.2 pd.merge(): A Lógica de Banco de Dados

`merge()` é a função para quando você precisa combinar `DataFrames` com base na correspondência de valores em uma ou mais colunas, conhecidas como “chaves” (`keys`). É análogo ao JOIN do SQL.

Tipos de Joins A forma como o `merge` combina os dados é definida pelo parâmetro `how`. A melhor maneira de visualizar isso é com um Diagrama de Venn, onde cada círculo representa um `DataFrame`.

```

[31]: import matplotlib.pyplot as plt
      from matplotlib.patches import Circle, Ellipse

      # --- Criação da Figura e Eixos ---
      # Configura a área de plotagem

```

```

fig, ax = plt.subplots(figsize=(10, 7))
ax.set_xlim(0, 10)
ax.set_ylim(0, 7)
ax.axis('off')
fig.suptitle('IMAGEM CONCEITUAL: Diagrama de Venn dos Tipos de Join',
             ↪ fontsize=16, weight='bold', y=0.9)

# --- Desenho dos Círculos ---
# Círculo da Esquerda (representa o DataFrame "Left")
left_circle = Circle((4, 3.5), 2, color='skyblue', alpha=0.6)
ax.add_patch(left_circle)
ax.text(3, 5.8, 'DataFrame A\n(Esquerda)', ha='center', fontsize=12)

# Círculo da Direita (representa o DataFrame "Right")
right_circle = Circle((6, 3.5), 2, color='salmon', alpha=0.6)
ax.add_patch(right_circle)
ax.text(7, 5.8, 'DataFrame B\n(Direita)', ha='center', fontsize=12)

# --- Legendas dos Tipos de Join ---

# Inner Join (A intersecção)
ax.text(5, 3.5, 'Inner Join', ha='center', va='center', fontsize=12,
       ↪ weight='bold', color='black')
ax.annotate('', xy=(5, 3.8), xytext=(5, 4.5),
            arrowprops=dict(facecolor='black', shrink=0.05, width=1,
                            ↪ headwidth=5))

# Left Join (Todo o círculo da esquerda)
ax.text(1.5, 1, 'Left Join\n(Todas as linhas de A)', ha='center', fontsize=12,
       ↪ bbox=dict(boxstyle="round,pad=0.5", fc="skyblue", ec="blue", alpha=0.8))

# Right Join (Todo o círculo da direita)
ax.text(8.5, 1, 'Right Join\n(Todas as linhas de B)', ha='center', fontsize=12,
       ↪ bbox=dict(boxstyle="round,pad=0.5", fc="salmon", ec="red", alpha=0.8))

# Outer Join (A união de ambos os círculos)
ax.text(5, 0.5, 'Outer Join (Todas as linhas de A e B)', ha='center',
       ↪ va='center', fontsize=12)

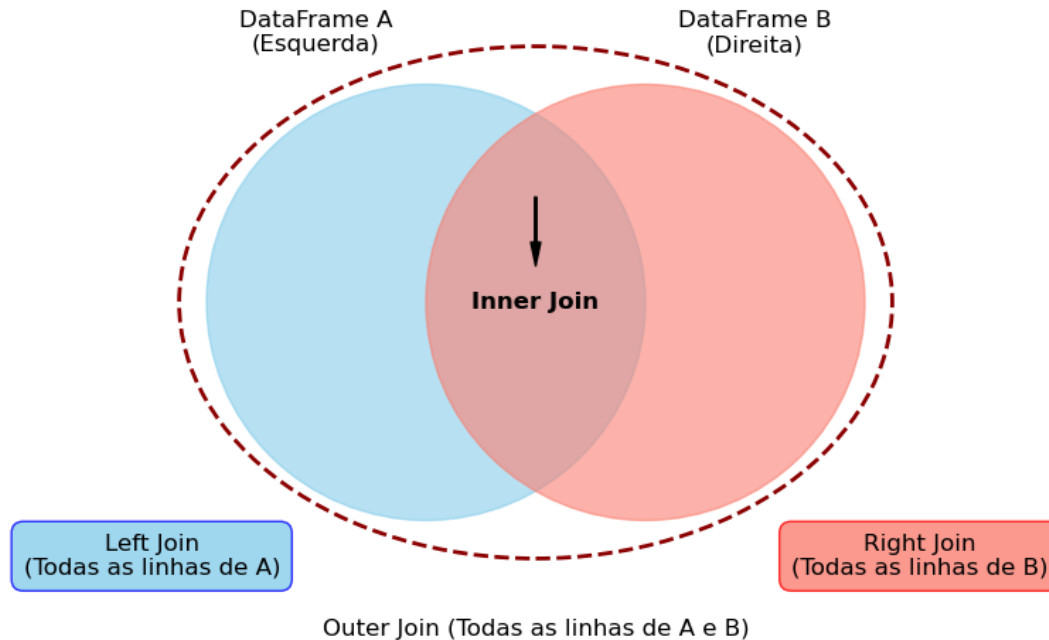
# Desenha uma elipse tracejada para indicar a união completa
union_ellipse = Ellipse((5, 3.5), 6.5, 4.7, fill=False, edgecolor='darkred',
                        ↪ lw=2, linestyle='--')
ax.add_patch(union_ellipse)

# --- Exibir a Imagem ---

```

```
plt.show()
```

IMAGEM CONCEITUAL: Diagrama de Venn dos Tipos de Join



[IMAGEM CONCEITUAL: Diagrama de Venn dos Tipos de Join]

Um diagrama de Venn com dois círculos sobrepostos. * A intersecção é o **Inner Join**. * O círculo da esquerda (incluindo a intersecção) é o **Left Join**. * O círculo da direita (incluindo a intersecção) é o **Right Join**. * A união de ambos os círculos é o **Outer Join**.

- `how='inner'` (padrão): Retorna apenas as linhas onde a chave de junção existe em **ambos** os DataFrames (a intersecção).
- `how='outer'`: Retorna **todas** as linhas de **ambos** os DataFrames. Se não houver correspondência para uma linha em um dos lados, as colunas do outro lado serão preenchidas com NaN.
- `how='left'`: Retorna **todas** as linhas do DataFrame da **esquerda** e as linhas correspondentes do DataFrame da direita. Se não houver correspondência na direita, suas colunas serão preenchidas com NaN.
- `how='right'`: O oposto do `left join`. Retorna **todas** as linhas do DataFrame da **direita** e as correspondentes da esquerda.

```
[32]: # Vamos criar dois DataFrames: um de funcionários e um de departamentos.  
funcionarios = pd.DataFrame({  
    'id_depto': [1, 2, 3, 4],  
    'Nome': ['Alice', 'Bob', 'Charles', 'Diane'],
```

```

        'Salario': [5000, 6000, 4500, 7200]
    })

departamentos = pd.DataFrame({
    'id_depto': [1, 2, 5],
    'NomeDepto': ['Vendas', 'T.I.', 'Marketing']
})

print("--- Funcionários ---")
print(funcionarios)
print("\n--- Departamentos ---")
print(departamentos)

```

```

--- Funcionários ---
   id_depto  Nome  Salario
0         1  Alice    5000
1         2   Bob    6000
2         3 Charles    4500
3         4  Diane    7200

```

```

--- Departamentos ---
   id_depto NomeDepto
0         1   Vendas
1         2     T.I.
2         5  Marketing

```

```

[33]: # Inner Join (padrão)
      # Une com base na coluna 'id_depto'
      # Charles (id 3), Diane (id 4) e Marketing (id 5) são excluídos por não terem
      # correspondência.
      inner_merge = pd.merge(funcionarios, departamentos, on='id_depto', how='inner')
      print("\n--- Inner Merge ---")
      print(inner_merge)

      # Outer Join
      # Todos são incluídos. NaN aparece onde não há correspondência.
      outer_merge = pd.merge(funcionarios, departamentos, on='id_depto', how='outer')
      print("\n--- Outer Merge ---")
      print(outer_merge)

      # Left Join
      # Todos os funcionários são mantidos. O departamento de Marketing é descartado.
      left_merge = pd.merge(funcionarios, departamentos, on='id_depto', how='left')
      print("\n--- Left Merge ---")
      print(left_merge)

      # Right Join

```

```
# Todos os departamentos são mantidos. Os funcionários Charles e Diane são
↳ descartados.
right_merge = pd.merge(funcionarios, departamentos, on='id_depto', how='right')
print("\n--- Right Merge ---")
print(right_merge)
```

--- Inner Merge ---

	id_depto	Nome	Salario	NomeDepto
0	1	Alice	5000	Vendas
1	2	Bob	6000	T.I.

--- Outer Merge ---

	id_depto	Nome	Salario	NomeDepto
0	1	Alice	5000.0	Vendas
1	2	Bob	6000.0	T.I.
2	3	Charles	4500.0	NaN
3	4	Diane	7200.0	NaN
4	5	NaN	NaN	Marketing

--- Left Merge ---

	id_depto	Nome	Salario	NomeDepto
0	1	Alice	5000	Vendas
1	2	Bob	6000	T.I.
2	3	Charles	4500	NaN
3	4	Diane	7200	NaN

--- Right Merge ---

	id_depto	Nome	Salario	NomeDepto
0	1	Alice	5000.0	Vendas
1	2	Bob	6000.0	T.I.
2	5	NaN	NaN	Marketing

1.4.3 4.3 DataFrame.join(): Junção por Índices

O `.join()` é um método do próprio `DataFrame`. É um atalho para o `merge` que, por padrão, une `DataFrames` com base em seus **índices**. O comportamento padrão é um `left join`.

É particularmente útil quando você tem dados relacionados em diferentes tabelas que compartilham o mesmo índice.

```
[34]: # Vamos usar DataFrames com índices significativos
dados_pessoais = pd.DataFrame({
    'Idade': [25, 30, 45],
    'Cidade': ['SP', 'RJ', 'BH']
}, index=['Alice', 'Bob', 'Eve'])

dados_profissionais = pd.DataFrame({
```

```

    'Salario': [5000, 8000],
    'Cargo': ['Analista', 'Gerente']
}, index=['Alice', 'Bob'])

print("--- Dados Pessoais ---")
print(dados_pessoais)
print("\n--- Dados Profissionais ---")
print(dados_profissionais)

# O .join() é chamado a partir do DataFrame da "esquerda"
# Por padrão, é um left join, então todos de 'dados_pessoais' são mantidos.
resultado_join = dados_pessoais.join(dados_profissionais)
print("\n--- Resultado do .join() (padrão left) ---")
print(resultado_join)

# Podemos especificar um inner join para manter apenas os índices comuns
resultado_inner_join = dados_pessoais.join(dados_profissionais, how='inner')
print("\n--- Resultado do .join() com how='inner' ---")
print(resultado_inner_join)

```

--- Dados Pessoais ---

	Idade	Cidade
Alice	25	SP
Bob	30	RJ
Eve	45	BH

--- Dados Profissionais ---

	Salario	Cargo
Alice	5000	Analista
Bob	8000	Gerente

--- Resultado do .join() (padrão left) ---

	Idade	Cidade	Salario	Cargo
Alice	25	SP	5000.0	Analista
Bob	30	RJ	8000.0	Gerente
Eve	45	BH	NaN	NaN

--- Resultado do .join() com how='inner' ---

	Idade	Cidade	Salario	Cargo
Alice	25	SP	5000	Analista
Bob	30	RJ	8000	Gerente

1.5 Conclusão

- Começamos com as estruturas de dados **Series** e **DataFrame**.
- Aprendemos a **manipular colunas**, uma tarefa diária na preparação de dados.
- Mergulhamos fundo no conceito de **groupby**, a ferramenta essencial para análises segmentadas, explorando a estratégia **Dividir-Aplicar-Combinar** através de agregação, transformação e

filtragem.

- Finalmente, desvendamos as três principais formas de **combinar DataFrames**:
 - **concat**: Para empilhar dados de forma simples.
 - **merge**: Para junções complexas baseadas em chaves, com a lógica de bancos de dados.
 - **join**: Um atalho conveniente para junções baseadas em índices.